

POLITECNICO DI TORINO

Master of Science in Computer Engineering / Data Science

Large Language Models (Large Language Models for Software Engineering)**Official Examination Session – February 11, 2025**

Duration: 75 minutes (14:11 – 15:26) | Total Available Points: 18.00 | Number of Questions: 12

Instructions to Candidates:

- The examination consists of 12 questions spanning foundational language modeling, transformer mechanics, agentic workflows, quantization, and software engineering evaluation metrics.
- Carefully note penalty rules: questions with penalties subtract 25% of the question's base score per incorrect choice. Questions marked with "no penalty" do not penalize incorrect attempts.
- In Part I, the complete, unabridged exam questions as presented to students are recorded.
- In Part II, comprehensive, production-grade solutions, mathematical derivations, error diagnoses, and theoretical commentaries are provided.

1 Part I: Exam Questions**Question 1****[2.00 Points | No Penalty]**

What is the likely output of the following code snippet?

Provide a brief explanation (max 50 words).

If an error occurs, indicate the line number and briefly explain why.

```
1 from transformers import AutoModel, AutoTokenizer
2
3 model_name = "bert-base-uncased"
4 model = AutoModel.from_pretrained(model_name)
5 tokenizer = AutoTokenizer.from_pretrained(model_name)
6
7 prompt = "The result of the operation 2 + 2 is "
8 tokens = tokenizer(prompt, return_tensors="pt")
9
10 result = model.generate(**tokens)
11 print(tokenizer.batch_decode(result))
```

```
1 from transformers import AutoModel, AutoTokenizer
2
3 model_name = "bert-base-uncased"
4 model = AutoModel.from_pretrained(model_name)
5 tokenizer = AutoTokenizer.from_pretrained(model_name)
6
7 prompt = "The result of the operation 2 + 2 is "
8 tokens = tokenizer(prompt, return_tensors="pt")
9
10 result = model.generate(**tokens)
11 print(tokenizer.batch_decode(result))
12
```

Question 2**[1.00 Point | Penalty: -0.25 Points]**

What is the purpose of reflection in an agentic system?

- a. To eliminate the need for structured workflows
- b. To generate human-like responses using predefined scripts
- c. To store knowledge permanently in short-term memory
- d. To execute tasks without modifying strategies
- e. To evaluate past decisions and improve future performance

Question 3**[1.00 Point | Penalty: -0.25 Points]**

What is the main difference between encoder-decoder cross-attention and encoder self-attention?

- a. In encoder-decoder cross-attention, the decoder attends only to the current token in the input sequence, while in encoder self-attention, the encoder attends to all tokens in the input
- b. Encoder self-attention attends to the encoder's output, while encoder-decoder cross-attention attends to the decoder's previous tokens
- c. In encoder self-attention, the decoder attends to the input sequence, whereas in encoder-decoder cross-attention, the encoder attends to the output sequence
- d. Encoder self-attention is used in the decoder, and encoder-decoder cross-attention is used in the encoder
- e. None of the other options is correct
- f. Encoder-decoder cross-attention attends only to the previous tokens in the output sequence, while encoder self-attention attends only to tokens in the input sequence
- g. Encoder self-attention only uses information from the input sequence, while encoder-decoder cross-attention allows the decoder to attend to the encoder's output

Question 4**[1.50 Points | No Penalty]**

You are given the following sequence of characters:

a c e a c e a a a a c b

Initially, each character is a separate token.

Apply Byte-Pair Encoding (BPE) to produce 3 additional tokens. What are the generated tokens?

Write each token, in the correct order, in the following slots.

Separate the merged tokens with a comma (and no spaces).

Examples:

- 1) If the merged tokens are **aa** and **b**, write: **aa,b**
- 2) If the merged tokens are **x** and **y**, write: **x,y**

1st merge: _____

2nd merge: _____

3rd merge: _____

Question 5**[1.00 Point | Penalty: -0.25 Points]**

Which of the following best describes how BERTScore evaluates text similarity?

- a. It measures sequence similarity using n -gram co-occurrence weighted by IDF scores
- b. It uses a fine-tuned BERT model to classify whether the reference and candidate texts are

semantically related or not

- c. It predicts a similarity score by treating the evaluation task as a masked language modeling problem, where BERT predicts the score as the next token
- d. It assigns similarity scores based on syntactic parse trees rather than token embeddings
- e. It uses cosine similarity on static word embeddings trained with Word2Vec or GloVe
- f. It applies a bag-of-words model to compare token frequency distributions between reference and candidate texts
- g. It computes token-level similarity by aligning contextualized embeddings from a pre-trained transformer model and aggregating scores

Question 6

[2.00 Points | No Penalty]

Apply **absmax quantization** to the following values x_1, x_2, x_3, x_4 , to map them to signed 8-bit integers (`int8`):

$$x_1 = -5$$

$$x_2 = 4$$

$$x_3 = -1$$

$$x_4 = 0$$

Write the quantized integer versions in the following boxes:

quantized(x_1) = _____

quantized(x_2) = _____

quantized(x_3) = _____

quantized(x_4) = _____

Question 7

[1.00 Point | Penalty: -0.25 Points per wrong selection]

Which of the following are benefits of using open-source datasets for training LLMs?

- a. Authenticity and credibility of real-world data
- b. Guaranteed high accuracy in model predictions
- c. Ability to acquire timely data and share data freely
- d. Elimination of bias in training data
- e. LLMs no longer require fine-tuning

Question 8

[3.00 Points | No Penalty]

Consider a scenario where you have:

- A natural language description of a function's requirements (**REQS**)
- A set of test cases (**TESTS**) specifying expected inputs and outputs

You need to design an agentic architecture using LLaMA-based agents to:

1. Generate an initial implementation of the function based on **REQS**
2. Refine the code iteratively until all test cases pass

Describe the architecture of LLaMA agents that perform these steps. For each agent:

- Define its role in the pipeline
- Specify its prompt, including:
 - **System message:** Defines the agent's behavior
 - **Context:** Information given to the agent from previous steps

- **Possible inputs:** Data received from prior agents
- **Expected output:** The structured result the agent should return

Notice: It is not necessary to write Python calls to transformers, code for output parsing, or actual code for test execution and parsing. Focus on the agentic workflow and prompt engineering techniques.

Question 9**[2.00 Points | No Penalty]**

Consider the following cosine similarity table between actual software requirements/artifacts (Act1–Act4) and expected artifacts (Exp1–Exp4). The green-shaded cells along the diagonal represent the ground-truth valid trace links (positive instances):

	Exp1	Exp2	Exp3	Exp4
Act1	0,71	0,65	0,62	0,30
Act2	0,71	0,74	0,30	0,20
Act3	0,54	0,05	0,60	0,55
Act4	0,80	0,40	0,15	0,92

	Exp1	Exp2	Exp3	Exp4
Act1	0.71	0.65	0.62	0.30
Act2	0.71	0.74	0.30	0.20
Act3	0.54	0.05	0.60	0.55
Act4	0.80	0.40	0.15	0.92

Which of the following thresholds is more dependable in terms of True Positive Rate (TPR) and False Positive Rate (FPR)?

Candidate Thresholds: $T = 0.70, \quad 0.80, \quad 0.90$

Provide full derivations of True Positives (TP), False Positives (FP), True Negatives (TN), False Negatives (FN), TPR, and FPR for each threshold, and justify your conclusion.

Question 10**[1.00 Point | Penalty: −0.25 Points per wrong selection]**

Which solutions can help address the challenge of variation in extracted requirements?

- Automatically rejecting extracted requirements with different phrasing
- Ignoring minor variations and focusing only on exact matches
- Using predefined synonym lists to map similar terms
- Using word embeddings to detect semantically similar words
- Manually reviewing every extracted requirement

Question 11**[1.50 Points | Penalty: −0.375 Points]**

The following reward formulation can be used when fine-tuning a model on human feedback:

$$R(x, y) = r_{\theta}(x, y) - \beta \mathbb{D}_{\text{KL}}(\pi_{\phi}(y | x) \| \pi_{\text{ref}}(y | x))$$

which can be expressed at the token or sequence level with the penalty term $-\beta \log \frac{\pi_{\phi}(y|x)}{\pi_{\text{ref}}(y|x)}$. Here, $r_{\theta}(x, y)$ is the output of a Reward Model, and π_{ϕ} and π_{ref} are the new and the original policies, respectively (representing the token probabilities assigned by the new and original models).

What is the purpose of the term $\log \frac{\pi_\phi(y|x)}{\pi_{\text{ref}}(y|x)}$?

- a. It increases the entropy of the output (i.e., it helps the model produce more “creative” answers)
- b. It improves the model’s capability of following instructions
- c. It prevents the model from collapsing into a model that always predicts a single token
- d. It helps make the model more deterministic
- e. None of the other answers is correct
- f. It prevents the new model from producing outputs that are too dissimilar from the original one
- g. It rewards the model when it generates an answer that humans will “like”

Question 12**[1.00 Point | Penalty: −0.25 Points per wrong selection]**

What is “priming” in the context of prompt engineering?

- a. Using reinforcement learning to fine-tune the model
- b. Pre-training the model with additional data
- c. Providing initial input to guide the model’s response
- d. Adding noise to the input to test model robustness
- e. Resetting the model before every new prompt

2 Part II: Comprehensive Solutions & In-Depth Analysis

Solution to Question 1: Debugging BERT Generation & Encoder Limitations

Concise Exam Answer:

The code crashes with an `AttributeError` at Line 10 (`result = model.generate(**tokens)`). `AutoModel` instantiates a bare `BertModel`, which lacks a generation head and does not implement `.generate()`. Furthermore, BERT is an encoder-only architecture trained bidirectionally on masked language modeling, not an autoregressive causal decoder capable of next-token generation.

1. Runtime Mechanics and Exact Exception

When executing the provided script:

```
model = AutoModel.from_pretrained("bert-base-uncased")
...
result = model.generate(**tokens)
```

The Python runtime raises the following exception at Line 10:

```
AttributeError: 'BertModel' object has no attribute 'generate'
```

In the Hugging Face `transformers` library:

1. `AutoModel.from_pretrained(...)` loads the *bare* base transformer backbone (specifically `BertModel`), which outputs contextual hidden state representations $\mathbf{H} \in \mathbb{R}^{B \times L \times d_{\text{model}}}$ from the final self-attention layer. It has no output vocabulary head (unembedding matrix W_U) attached.
2. The autoregressive generation API method `.generate()` is provided by the mixin class `GenerationMixin`. In Hugging Face, `GenerationMixin` is exclusively inherited by model classes equipped with language modeling heads, such as `AutoModelForCausalLM` (e.g., `GPT2LMHeadModel`, `LlamaForCausalLM`) or `AutoModelForSeq2SeqLM` (e.g., `T5ForConditionalGeneration`, `BartForConditionalGeneration`). Bare backbone models like `BertModel` do *not* inherit from `GenerationMixin`.

2. Architectural and Theoretical Foundations: Encoder vs. Causal Decoder

Beyond the missing library method, the code snippet exhibits a fundamental conceptual mistake:

- **Bidirectional Encoder Architecture (BERT):** BERT (Devlin et al., 2018) is an *encoder-only* model. Its self-attention mechanism is entirely unmasked and bidirectional:

$$\mathbf{A}_{i,j} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}} \quad \forall i, j \in \{1, \dots, L\} \quad (1)$$

Every token representation attends simultaneously to all prior and subsequent tokens in the sequence. BERT is pre-trained on **Masked Language Modeling (MLM)** and **Next Sentence Prediction (NSP)**. In MLM, 15% of tokens are masked with `[MASK]` and predicted in parallel. BERT has no mechanism for step-by-step causal next-token sampling $P(w_t | w_{<t})$.

- **Autoregressive Causal Decoder (GPT / LLaMA):** Text generation requires an autoregressive decoder that enforces a lower-triangular causal attention mask:

$$\mathbf{M}_{i,j} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases} \quad (2)$$

This ensures that token w_t can only attend to past tokens $w_{<t}$, permitting valid probability factorizations $P(w_{1:T}) = \prod_{t=1}^T P(w_t | w_{<t})$ and token-by-token greedy or nucleus decoding.

3. How to Correct the Code

If the developer's objective is open-ended sequence generation (e.g., completing "The result of the operation $2 + 2$ is "), an autoregressive causal language model must be used:

```
1 from transformers import AutoModelForCausalLM, AutoTokenizer
2 import torch
3
4 model_name = "gpt2" # or a causal LLaMA checkpoint
5 tokenizer = AutoTokenizer.from_pretrained(model_name)
6 model = AutoModelForCausalLM.from_pretrained(model_name)
7
8 prompt = "The result of the operation 2 + 2 is "
9 tokens = tokenizer(prompt, return_tensors="pt")
10
11 with torch.no_grad():
12     output_tokens = model.generate(**tokens, max_new_tokens=10)
13 print(tokenizer.decode(output_tokens[0], skip_special_tokens=True))
```

If BERT was intended for Cloze token filling, one must instead use `AutoModelForMaskedLM` and supply a [MASK] token:

```
1 from transformers import AutoModelForMaskedLM, AutoTokenizer
2 tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
3 model = AutoModelForMaskedLM.from_pretrained("bert-base-uncased")
4 inputs = tokenizer("The result of the operation 2 + 2 is [MASK].",
5                     return_tensors="pt")
6 logits = model(**inputs).logits
7 mask_idx = (inputs.input_ids == tokenizer.mask_token_id).nonzero(as_tuple=
8                 True)[1]
9 predicted_token = tokenizer.decode(logits[0, mask_idx].argmax(dim=-1))
```

Solution to Question 2: Reflection Mechanisms in Agentic Architectures

Correct Choice: e ("To evaluate past decisions and improve future performance")

1. Theoretical Rationale & Literature Context

In agentic AI systems (e.g., REFLEXION by Shinn et al., 2023; SELF-REFINE by Madaan et al., 2023; REACT by Yao et al., 2022), **reflection** constitutes a metacognitive feedback loop. Rather than operating in an open-loop feed-forward manner (where an error irrevocably derails the task), an agent with reflection:

1. **Evaluates Trajectories:** Analyzes execution feedback from the environment (e.g., unit test failures, compiler syntax errors, linter diagnostics, or human critiques).
2. **Formulates Verbal Reflections:** Generates a semantic diagnosis of why the previous attempt failed and what specific tactical modifications are required.
3. **Updates Memory:** Stores these synthesized reflections into episodic/working memory buffers, dynamically modifying the context for the next planning iteration.

This iterative self-correction dramatically improves performance on complex reasoning, theorem proving, and automated software repair benchmarks (such as SWE-bench and HumanEval).

2. Detailed Breakdown of Distractors

- **Option a is incorrect:** Reflection does not eliminate structured workflows. On the contrary, reflection relies upon rigorous structural loops (such as Actor–Evaluator–Self-Reflection state machines).

- **Option b is incorrect:** Generating responses using predefined scripts describes rule-based chatterbots, which are rigid and antithetical to autonomous LLM reasoning.
- **Option c is incorrect:** This statement contains an oxymoron: “store knowledge permanently in short-term memory”. Short-term memory (the active context window) is ephemeral; permanent storage occurs in external long-term vector databases.
- **Option d is incorrect:** Executing tasks without modifying strategies is the definition of non-adaptive execution. The foundational goal of reflection is specifically to adapt and revise failing strategies.

Solution to Question 3: Encoder Self-Attention vs. Cross-Attention

Correct Choice: g (“Encoder self-attention only uses information from the input sequence, while encoder-decoder cross-attention allows the decoder to attend to the encoder’s output”)

1. Mathematical Formulation

The scaled dot-product attention operator across all Transformer modules is defined as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (3)$$

The critical distinction lies in where the Query ($\mathbf{Q}$), Key ($\mathbf{K}$), and Value ($\mathbf{V}$) matrices originate:

Encoder Self-Attention: Let $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$ denote the input token representations (or hidden states from layer $l - 1$ of the encoder). All three projections originate from the *same* sequence:

$$\mathbf{Q}_{\text{enc}} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K}_{\text{enc}} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V}_{\text{enc}} = \mathbf{X}\mathbf{W}_V \quad (4)$$

Every token in the input sequence computes attention over all other tokens in the input sequence bidirectionally. It operates strictly within the source domain.

Encoder-Decoder Cross-Attention: Let $\mathbf{S} \in \mathbb{R}^{m \times d_{\text{model}}}$ denote the decoder’s intermediate representations (derived from the masked self-attention over previously generated target tokens), and let $\mathbf{H}_{\text{enc}} \in \mathbb{R}^{n \times d_{\text{model}}}$ denote the final output representations of the encoder stack. In cross-attention:

$$\mathbf{Q}_{\text{cross}} = \mathbf{S}\mathbf{W}_Q, \quad \mathbf{K}_{\text{cross}} = \mathbf{H}_{\text{enc}}\mathbf{W}_K, \quad \mathbf{V}_{\text{cross}} = \mathbf{H}_{\text{enc}}\mathbf{W}_V \quad (5)$$

Here, the decoder queries ($\mathbf{Q}$) probe the encoder’s memory representations ($\mathbf{K}, \mathbf{V}$). This allows the generative decoder to condition its next token predictions on the complete semantic context of the input sequence.

2. Detailed Breakdown of Distractors

- **Option a is incorrect:** Cross-attention attends to the *entire* input sequence (via all encoder key vectors $\mathbf{k}_1, \dots, \mathbf{k}_n$), not just a single “current token”.
- **Option b is incorrect:** Inverts the mechanisms: encoder self-attention attends to prior encoder layers, while cross-attention attends to encoder outputs using decoder queries.
- **Option c is incorrect:** States that encoder self-attention occurs in the decoder, which is completely backwards.
- **Option d is incorrect:** Inverts the architectural locations of the blocks.
- **Option e is incorrect:** Because option (g) is strictly correct.
- **Option f is incorrect:** Attending to previous tokens in the output sequence is the function of *masked decoder self-attention*, not cross-attention.

Solution to Question 4: Byte-Pair Encoding (BPE) Step-by-Step Derivation**Correct Formatted Answer:**

1st merge: a,a | 2nd merge: a,c | 3rd merge: ac,e

1. Step-by-Step Mathematical Derivation

Byte-Pair Encoding (Sennrich et al., 2016) is a greedy data compression algorithm adapted for subword tokenization. At each step, it identifies the most frequently co-occurring adjacent symbol pair across the corpus, merges them into a new single vocabulary symbol, and updates the tokenized sequence.

Initial State (Step 0): The initial sequence consists of 13 individual character tokens:

[a, c, e, a, c, e, a, a, a, a, a, c, b]

Let us enumerate all adjacent symbol pairs and compute their frequencies:

- Pair ('a', 'c'): indices (0,1), (3,4), (11,12) $\Rightarrow$ **Frequency = 3**
- Pair ('c', 'e'): indices (1,2), (4,5) $\Rightarrow$ **Frequency = 2**
- Pair ('e', 'a'): indices (2,3), (5,6) $\Rightarrow$ **Frequency = 2**
- Pair ('a', 'a'): in the sub-sequence a a a a a (indices 6 to 10), adjacent pairs appear at (6,7), (7,8), (8,9), (9,10) $\Rightarrow$ **Frequency = 4**
- Pair ('c', 'b'): index (11,12) $\Rightarrow$ **Frequency = 1**

The maximum frequency is $\max = 4$, corresponding uniquely to the pair ('a', 'a').

Merge 1 Selection

1st merge: a,a $\rightarrow$ generates new token aa.

Replacing all non-overlapping occurrences of a a from left to right in a a a a a:

- Tokens at 6,7 $\rightarrow$ aa
- Tokens at 8,9 $\rightarrow$ aa
- Token at 10 remains single a

Updated sequence after Merge 1 (11 tokens):

[a, c, e, a, c, e, aa, aa, a, c, b]

State after Merge 1 (Step 1): Let us re-evaluate the frequencies of all adjacent pairs:

- Pair ('a', 'c'): indices (0,1), (3,4), and between the single a at index 8 and c at index 9: (8,9) $\Rightarrow$ **Frequency = 3**
- Pair ('c', 'e'): indices (1,2), (4,5) $\Rightarrow$ **Frequency = 2**
- Pair ('e', 'a'): index (2,3) $\Rightarrow$ **Frequency = 1**
- Pair ('e', 'aa'): index (5,6) $\Rightarrow$ **Frequency = 1**
- Pair ('aa', 'aa'): index (6,7) $\Rightarrow$ **Frequency = 1**
- Pair ('aa', 'a'): index (7,8) $\Rightarrow$ **Frequency = 1**
- Pair ('c', 'b'): index (9,10) $\Rightarrow$ **Frequency = 1**

The most frequent adjacent pair is strictly ('a', 'c') with frequency 3.

Merge 2 Selection

2nd merge: a,c $\rightarrow$ generates new token ac.

Updated sequence after Merge 2 (8 tokens):

[ac, e, ac, e, aa, aa, ac, b]

State after Merge 2 (Step 2): Let us re-evaluate the adjacent pair frequencies:

- Pair ('ac', 'e'): indices (0,1) and (2,3) $\implies$ **Frequency = 2**
- Pair ('e', 'ac'): index (1,2) $\implies$ **Frequency = 1**
- Pair ('e', 'aa'): index (3,4) $\implies$ **Frequency = 1**
- Pair ('aa', 'aa'): index (4,5) $\implies$ **Frequency = 1**
- Pair ('aa', 'ac'): index (5,6) $\implies$ **Frequency = 1**
- Pair ('ac', 'b'): index (6,7) $\implies$ **Frequency = 1**

The highest frequency is 2, corresponding uniquely to ('ac', 'e').

Merge 3 Selection

3rd merge: ac,e $\longrightarrow$ generates new token ace.

Final sequence after Merge 3 (6 tokens):

[ace, ace, aa, aa, ac, b]

The three merged tokens produced, in exact sequential order, are **a,a,a,c**, and **ac,e**.

Solution to Question 5: Principles of BERTScore Evaluation

Correct Choice: g ("It computes token-level similarity by aligning contextualized embeddings from a pre-trained transformer model and aggregating scores")

1. Theoretical Rationale and Mathematical Formalism

Traditional text generation metrics like BLEU and ROUGE rely on surface lexical n -gram matching. Consequently, they penalize semantically accurate paraphrases, synonyms, and reordered syntax. **BERTScore** (Zhang et al., ICLR 2020) solves this limitation by evaluating semantic similarity in continuous representation space:

1. Given a candidate sentence $\hat{x} = \langle \hat{x}_1, \dots, \hat{x}_k \rangle$ and a reference sentence $x = \langle x_1, \dots, x_l \rangle$, BERTScore passes them through a pre-trained contextual transformer (e.g., RoBERTa, BERT) to extract contextualized token embeddings:

$$\hat{\mathbf{e}}_i = \frac{\text{BERT}(\hat{x})_i}{\|\text{BERT}(\hat{x})_i\|_2}, \quad \mathbf{e}_j = \frac{\text{BERT}(x)_j}{\|\text{BERT}(x)_j\|_2} \quad (6)$$

2. Pairwise cosine similarities between all tokens are computed: $S_{i,j} = \hat{\mathbf{e}}_i^\top \mathbf{e}_j$.
3. Greedy bipartite matching is applied to compute **BERTScore Recall** and **Precision**:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_j \in x} \max_{\hat{x}_i \in \hat{x}} \hat{\mathbf{e}}_i^\top \mathbf{e}_j, \quad P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{\hat{x}_i \in \hat{x}} \max_{x_j \in x} \hat{\mathbf{e}}_i^\top \mathbf{e}_j \quad (7)$$

4. The final metric is the harmonic mean: $F_{\text{BERT}} = 2 \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}}.$

2. Detailed Breakdown of Distractors

- **Option a is incorrect:** Describes n -gram overlap metrics (like METEOR or tf-idf weighted BLEU), not BERTScore's contextualized vector alignment.
- **Option b is incorrect:** BERTScore does not use a fine-tuned binary classifier or cross-encoder; it utilizes frozen, pre-trained embeddings extracted from intermediate/final layers.
- **Option c is incorrect:** The candidate in the exam dump mistakenly selected this option. BERTScore does *not* formulate evaluation as a masked language modeling fill-in-the-blank prompt.

- **Option d is incorrect:** BERTScore is entirely based on continuous vector dot-products; it does not construct or traverse discrete syntactic parse trees.
- **Option e is incorrect:** Static word embeddings (Word2Vec, GloVe) assign one fixed vector per word, completely failing to handle polysemy, syntactic role shifts, and contextual dependencies.
- **Option f is incorrect:** Bag-of-words ignores word ordering and context entirely.

Solution to Question 6: Absmax Quantization to int8

Correct Formatted Answer:

$$\text{quantized}(x_1) = -127$$

$$\text{quantized}(x_2) = 102$$

$$\text{quantized}(x_3) = -25$$

$$\text{quantized}(x_4) = 0$$

1. Mathematical Derivation of Symmetric Absmax Quantization

In post-training quantization of LLM weights and activations (Dettmers et al., LLM.int8(), 2022), **absmax quantization** maps full-precision floating point numbers $X = \{x_1, \dots, x_N\}$ into signed 8-bit integers $\text{int8} \in [-127, 127]$ using a symmetric scale factor.

Step 1: Determine the Absolute Maximum ($x_{\max}$):

$$x_{\max} = \max_i |x_i| = \max(|-5|, |4|, |-1|, |0|) = \max(5, 4, 1, 0) = 5 \quad (8)$$

Step 2: Calculate the Quantization Scale Factor (s): For signed symmetric 8-bit integers with dynamic range $[-127, 127]$:

$$s = \frac{q_{\max}}{x_{\max}} = \frac{127}{5} = 25.4 \quad (9)$$

Step 3: Quantize Each Value via Scaling and Rounding: The quantization formula is:

$$q_i = \text{round}(s \cdot x_i) = \text{round}(25.4 \cdot x_i) \quad (10)$$

Applying this to each input coordinate:

- For $x_1 = -5$:

$$q_1 = \text{round}(25.4 \times (-5)) = \text{round}(-127.0) = -127$$

- For $x_2 = 4$:

$$q_2 = \text{round}(25.4 \times 4) = \text{round}(101.6) = 102$$

- For $x_3 = -1$:

$$q_3 = \text{round}(25.4 \times (-1)) = \text{round}(-25.4) = -25$$

- For $x_4 = 0$:

$$q_4 = \text{round}(25.4 \times 0) = \text{round}(0.0) = 0$$

2. Dequantization and Error Analysis

To reconstruct the floating point approximation $\hat{x}_i$, the inverse transformation $\hat{x}_i = \frac{q_i}{s}$ is applied:

$$\hat{x}_1 = \frac{-127}{25.4} = -5.000 \quad (\text{Quantization error} = 0.000)$$

$$\hat{x}_2 = \frac{102}{25.4} \approx 4.0157 \quad (\text{Quantization error} = |4 - 4.0157| \approx 0.0157)$$

$$\hat{x}_3 = \frac{-25}{25.4} \approx -0.9843 \quad (\text{Quantization error} = |-1 - (-0.9843)| \approx 0.0157)$$

$$\hat{x}_4 = \frac{0}{25.4} = 0.000 \quad (\text{Quantization error} = 0.000)$$

This achieves a $4\times$ memory reduction (from 32-bit floats to 8-bit integers) with negligible numerical degradation.

Solution to Question 7: Benefits of Open-Source Datasets

Correct Choices: a and c

- **a.** Authenticity and credibility of real-world data
- **c.** Ability to acquire timely data and share data freely

1. Theoretical Rationale

- **Authenticity of Real-World Data (Option a):** Open web crawls (e.g., Common Crawl, Wikipedia, GitHub, StackOverflow, RedPajama) reflect natural human language, diverse linguistic idioms, real-world software engineering paradigms, and actual domain-specific jargon that synthetic data often lacks.
- **Timely Acquisition and Free Sharing (Option c):** Open-source licensing permits public distribution, rapid community dissemination, collective auditing, academic reproducibility, and continuous updates as new libraries and software technologies emerge.

2. Detailed Breakdown of Distractors

- **Option b is false:** No dataset guarantees high model prediction accuracy. Accuracy depends on model parameter capacity, loss convergence, algorithmic design, data cleaning, and domain alignment. In fact, raw open-source web scrapes contain substantial noise, spam, and factual falsehoods.
- **Option d is false:** Open-source data extracted from the internet is notorious for containing historical, cultural, gender, and toxic societal biases. Openness does *not* eliminate bias; rigorous deduplication, toxicity filtering, and alignment are mandatory.
- **Option e is false:** Pre-training on open-source text yields a foundational base model. Downstream task execution, instruction adherence, and safety guardrails strictly require supervised fine-tuning (SFT) and preference alignment (RLHF/DPO).

Solution to Question 8: Agentic Multi-Agent Architecture for Automated Code Generation and Iterative Test Refinement

1. Critique of Candidate Submission in Raw Exam Dump

In the recorded examination attempt, the student proposed an agent that “tweaks the tests”. As the course examiner correctly annotated:

“The objective of the second agent would not be to tweak the tests but to refine the code to pass a set of stable test cases.”

In automated software engineering (LLM4SE), test cases constitute the formal specification and ground-truth acceptance oracle. Modifying tests to match buggy code destroys software reliability and violates verification guarantees. Below is the production-grade, theoretically sound agentic architecture.

2. System Architecture & Workflow Pipeline

The system implements a closed-loop iterative repair loop (analogous to REFLEXION and SELF-DEBUGGING):

1. **Code Generator Agent (Architect):** Receives natural language requirements (REQS) and test signatures, generating an initial candidate implementation C_0 .
2. **Deterministic Execution Sandbox (Tool Environment):** Executes C_t against the immutable test suite TESTS. Returns execution status: either PASSED (terminates pipeline) or FAILED with structured failure logs (assertion messages, stack traces, stderr).
3. **Code Refiner Agent (Debugger):** When tests fail, this agent analyzes the failing inputs, expected vs. actual outputs, and tracebacks. Using Chain-of-Thought (CoT) reasoning, it identifies the bug and outputs an updated implementation C_{t+1} .
4. **Loop Controller:** Iterates until all tests pass or the maximum iteration budget ($T_{\max} = 5$) is exhausted.



3. Comprehensive Specification for Agent 1: Code Generator Agent

- **Role in Pipeline:** Analyzes the requirements specification REQS and produces a syntactically correct, self-contained Python implementation of the target function.
- **System Message:**

You are an expert Python Software Engineer. Your role is to generate clean, robust, and PEP-8 compliant code implementing the exact functionality specified in the provided requirements. You must return only the complete Python function enclosed in `“python ... “` blocks, with no conversational filler.

- **Context:** The natural language functional requirements (REQS) and the test case signatures/specifications (TESTS).
- **Possible Inputs:**

```

REQS: "Implement a function 'parse_version(v: str) -> tuple' that parses a
       semantic version string (e.g. '1.10.2') into a tuple of integers. It
       must handle pre-release tags by ignoring any suffix after a hyphen."
TESTS: [
  {"input": "'1.2.3'", "expected": "(1, 2, 3)"},
  {"input": "'2.0.0-alpha'", "expected": "(2, 0, 0)"}
]
  
```

- **Expected Output:**

```

“python
def parse_version(v: str) -> tuple:
    clean_v = v.split('-')[0]
    return tuple(int(x) for x in clean_v.split('.'))
“
  
```

4. Comprehensive Specification for Agent 2: Code Refiner Agent

- **Role in Pipeline:** Acts as an automated debugger. It evaluates the current buggy implementation alongside failing test traces, identifies the root semantic bug, and synthesizes a corrected implementation without regressing passing tests.
- **System Message:**

```
You are an expert Automated Program Repair (APR) agent. You are given a
function requirement, the current candidate code, and detailed execution
feedback showing failing test cases.
Analyze the bug step-by-step:
1. Explain why the current implementation failed on the failing test cases.
2. Formulate the precise algorithmic fix required.
3. Output the fully corrected, complete replacement function inside '''
python ... ''' blocks.
Never alter or complain about the test cases; they are ground truth.
```

- **Context:** Original requirements REQS, immutable test suite TESTS, current implementation C_t , and full execution traceback logs from the deterministic sandbox.
- **Possible Inputs:**

```
CURRENT_CODE:
def parse_version(v: str) -> tuple:
    return tuple(int(x) for x in v.split('.'))

EXECUTION_LOGS:
Test 1 ('1.2.3'): PASSED
Test 2 ('2.0.0-alpha'): FAILED -> ValueError: invalid literal for int()
with base 10: '0-alpha'
```

- **Expected Output:**

```
### Root Cause Analysis:
The function splits the input directly by '.' without stripping optional
pre-release suffixes denoted by '-'. Consequently, '0-alpha' cannot be
cast to an integer.

### Corrected Code:
'''python
def parse_version(v: str) -> tuple:
    base_version = v.split('-')[0]
    return tuple(int(part) for part in base_version.split('.'))
'''
```

Solution to Question 9: Similarity Matrix, Threshold Optimization, and ROC Analysis

Correct Answer: The threshold $T = 0.70$ is the most dependable threshold.

1. Problem Setup and Ground-Truth Identification

We are given a 4×4 similarity matrix evaluated across 4 actual artifacts (Act1–Act4) and 4 expected artifacts (Exp1–Exp4), representing a total of $4 \times 4 = 16$ candidate trace links.

	Exp1	Exp2	Exp3	Exp4
Act1	0.71	0.65	0.62	0.30
Act2	0.71	0.74	0.30	0.20
Act3	0.54	0.05	0.60	0.55
Act4	0.80	0.40	0.15	0.92

Ground-Truth Partitioning:

- **True Positives Class ($P = 4$):** The diagonal elements (highlighted green in the exam figure) represent the valid ground-truth trace links:

$$\mathcal{P} = \{(\text{Act1}, \text{Exp1}) = 0.71, (\text{Act2}, \text{Exp2}) = 0.74, \\ (\text{Act3}, \text{Exp3}) = 0.60, (\text{Act4}, \text{Exp4}) = 0.92\}$$

- **True Negatives Class ($N = 12$):** The remaining 12 off-diagonal elements represent non-matching artifact pairs:

$$\mathcal{N} = \{0.65, 0.62, 0.30, \quad (\text{Row 1}) \\ 0.71, 0.30, 0.20, \quad (\text{Row 2}) \\ 0.54, 0.05, 0.55, \quad (\text{Row 3}) \\ 0.80, 0.40, 0.15 \quad (\text{Row 4})\}$$

Note that the universe of links is $|\mathcal{P}| + |\mathcal{N}| = 4 + 12 = 16$.

Classification Metric Definitions: For a similarity threshold T , a candidate link is predicted positive if $\text{sim} \geq T$, and negative otherwise:

$$\text{TPR} = \frac{\text{TP}}{P} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{N} = \frac{\text{FP}}{\text{FP} + \text{TN}} \quad (11)$$

2. Step-by-Step Derivations for Each Threshold**Case 1: Threshold $T = 0.70$**

- **Evaluating Positives ($\mathcal{P}$):**

- $(\text{Act1}, \text{Exp1}) = 0.71 \geq 0.70 \implies \text{TP}$
 - $(\text{Act2}, \text{Exp2}) = 0.74 \geq 0.70 \implies \text{TP}$
 - $(\text{Act3}, \text{Exp3}) = 0.60 < 0.70 \implies \text{FN}$
 - $(\text{Act4}, \text{Exp4}) = 0.92 \geq 0.70 \implies \text{TP}$
- $\implies \mathbf{TP = 3}, \quad \mathbf{FN = 1} \quad (TP + FN = 4).$

$$\mathbf{TPR} = \frac{3}{4} = \mathbf{0.75} \quad (75.0\%)$$

- **Evaluating Negatives ($\mathcal{N}$):** Pairs with similarity ≥ 0.70 :

- $(\text{Act2}, \text{Exp1}) = 0.71 \geq 0.70 \implies \text{FP}$
- $(\text{Act4}, \text{Exp1}) = 0.80 \geq 0.70 \implies \text{FP}$

All other 10 off-diagonal values are strictly below 0.70 (0.65, 0.62, 0.30, 0.30, 0.20, 0.54, 0.05, 0.55, 0.40, 0.15). $\implies \mathbf{FP = 2}, \quad \mathbf{TN = 10} \quad (FP + TN = 12).$

$$\mathbf{FPR} = \frac{2}{12} = \frac{1}{6} \approx \mathbf{0.1667} \quad (16.67\%)$$

Case 2: Threshold $T = 0.80$

- **Evaluating Positives ($\mathcal{P}$):** Only $(\text{Act4}, \text{Exp4}) = 0.92 \geq 0.80$. The remaining three (0.71, 0.74, 0.60) are < 0.80 . $\implies \mathbf{TP = 1}, \quad \mathbf{FN = 3}.$

$$\mathbf{TPR} = \frac{1}{4} = \mathbf{0.25} \quad (25.0\%)$$

- **Evaluating Negatives ($\mathcal{N}$):** Among the 12 negative pairs, only $(\text{Act4}, \text{Exp1}) = 0.80 \geq 0.80 \implies \text{FP}$. The other 11 values are < 0.80 . $\implies \mathbf{FP = 1}, \quad \mathbf{TN = 11}.$

$$\mathbf{FPR} = \frac{1}{12} \approx \mathbf{0.0833} \quad (8.33\%)$$

(Note: If strict inequality $\text{sim} > 0.80$ is assumed, $FP = 0, TN = 12 \implies FPR = 0$, but TPR remains unacceptably low at 0.25).

Case 3: Threshold $T = 0.90$

- **Evaluating Positives ($\mathcal{P}$):** Only $(\text{Act4}, \text{Exp4}) = 0.92 \geq 0.90 \implies \mathbf{TP = 1, FN = 3.}$

$$\mathbf{TPR} = \frac{1}{4} = \mathbf{0.25} \quad (25.0\%)$$

- **Evaluating Negatives ($\mathcal{N}$):** No off-diagonal element satisfies $\text{sim} \geq 0.90$ (the maximum off-diagonal value is 0.80). $\implies \mathbf{FP = 0, TN = 12.}$

$$\mathbf{FPR} = \frac{0}{12} = \mathbf{0.00} \quad (0.0\%)$$

3. Comprehensive Synthesis & Dependability Justification

Threshold (T)	TP	FP	TN	FN	TPR (Recall)	FPR	Precision	F_1 -Score
$T = 0.70$	3	2	10	1	0.750	0.167	0.600	0.667
$T = 0.80$	1	1	11	3	0.250	0.083	0.500	0.333
$T = 0.90$	1	0	12	3	0.250	0.000	1.000	0.400

Why $T = 0.70$ is the most dependable:

1. **Asymmetry of Costs in Requirements Traceability:** In software engineering verification and trace link recovery, a **False Negative** (missed link) means an architectural component or unit test is completely detached from its functional requirement, leading to undetected compliance failures or untested safety critical functions. Conversely, a **False Positive** merely incurs a minor human verification cost.
2. **Catastrophic Recall Degradation at Higher Thresholds:** Increasing T from 0.70 to 0.80 drops the True Positive Rate from 75% to an abysmal 25% (missing 3 out of 4 requirements links).
3. **Optimal Balance on ROC Curve:** At $T = 0.70$, the model achieves a high TPR of 0.75 while bounding the False Positive Rate to a modest 16.7% (2 false alarms out of 12 negative links). It achieves the highest F_1 -score (0.667).

Examiner Note on Student Derivation in Raw Exam Dump: The candidate in the raw exam scored only 1.00/2.00 because while their final selection ($T = 0.70$) was correct, their intermediate calculations contained critical errors:

- At $T = 0.70$, the student wrote $TP = 3, FN = 2 \implies TPR = 3/5 = 0.6$ and $TN = 10, FP = 1 \implies FPR = 1/11 = 0.09$. The candidate hallucinated 5 positive links instead of 4 and missed an off-diagonal FP.
- At $T = 0.80$, the student wrote $TP = 1, FN = 0 \implies TPR = 1/1 = 1.0$, which is mathematically absurd since 3 positive links fell below the 0.80 threshold.

Solution to Question 10: Mitigating Linguistic Variation in Requirements

Correct Choices: c and d

- **c.** Using predefined synonym lists to map similar terms
- **d.** Using word embeddings to detect semantically similar words

1. Theoretical Rationale

In Natural Language Requirements Engineering (NLRE), the **vocabulary mismatch problem** arises because different stakeholders, systems, and specifications express identical functional behaviors using disparate terminology (e.g., “user”, “client”, “subscriber”; or “terminate”,

“abort”, “cancel”). Two effective AI/NLP strategies resolve this ambiguity:

- **Controlled Ontologies and Synonym Lists (Option c):** Using curated domain dictionaries or WordNet synsets allows automated pipelines to canonicalize disparate surface tokens to a standardized root concept.
- **Dense Word / Sentence Embeddings (Option d):** Continuous representation models (Word2Vec, GloVe, Sentence-BERT) embed lexical items into high-dimensional geometric spaces where semantic relatedness corresponds to vector proximity (high cosine similarity), enabling soft semantic matching that bridges synonymy and paraphrasing.

2. Detailed Breakdown of Distractors

- **Option a is incorrect:** Automatically rejecting requirements due to differing phrasing would discard valid stakeholder requirements, catastrophically reducing system recall.
- **Option b is incorrect:** Requiring exact lexical matches is brittle and fails completely on natural language variation.
- **Option e is incorrect:** Requiring manual human review for every requirement defeats the entire purpose of automated NLP extraction and does not scale to large software architectures.

Solution to Question 11: The KL Divergence Regularizer in RLHF

Correct Choice: f (“It prevents the new model from producing outputs that are too dissimilar from the original one”)

1. Mathematical Derivation of the RLHF Objective

In Reinforcement Learning from Human Feedback (RLHF) via Proximal Policy Optimization (PPO; Ziegler et al., 2019; Ouyang et al., InstructGPT, 2022), the policy π_ϕ is optimized against an empirical reward model $r_\theta(x, y)$ subject to a Kullback-Leibler (KL) divergence penalty against the frozen supervised fine-tuned (SFT) reference policy π_{ref} :

$$\max_{\phi} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\phi(\cdot|x)} [r_\theta(x, y) - \beta \mathbb{D}_{\text{KL}}(\pi_\phi(y|x) \parallel \pi_{\text{ref}}(y|x))] \quad (12)$$

Expanding the KL divergence for a discrete output sequence y :

$$\mathbb{D}_{\text{KL}}(\pi_\phi(y|x) \parallel \pi_{\text{ref}}(y|x)) = \sum_{t=1}^{|y|} \log \frac{\pi_\phi(y_t|x, y_{<t})}{\pi_{\text{ref}}(y_t|x, y_{<t})} \quad (13)$$

The penalty term $-\beta \log \frac{\pi_\phi}{\pi_{\text{ref}}}$ penalizes policy updates whenever π_ϕ places significantly higher probability mass on tokens than the original model π_{ref} .

2. Theoretical Necessity: Mitigating Reward Hacking and Policy Drift

1. **Preventing Reward Model Exploitation (Reward Hacking):** The reward model r_θ is a learned proxy and imperfect approximation of human judgment. In high-dimensional optimization, an unconstrained policy quickly discovers adversarial vulnerabilities or degenerate sequences (e.g., repetitive punctuation or specific buzzwords) that achieve artificially high reward scores despite being nonsensical to humans (Goodhart’s Law).
2. **Preventing Policy Collapse and Drift:** The KL regularizer anchors the policy to the distribution of natural, grammatically correct language learned during pre-training and SFT, preserving core reasoning and linguistic fluency.

3. Detailed Breakdown of Distractors

- **Option a is incorrect:** Policy entropy is governed by the entropy bonus $\mathcal{H}(\pi) = -\sum \pi \log \pi$. The KL divergence constrains the policy toward π_{ref} , which reduces rather than increases ungrounded randomness.
- **Option b is incorrect:** Instruction following capability is imparted by supervised instruction tuning (SFT) and the reward model scoring, not specifically by penalizing divergence from π_{ref} .
- **Option c is incorrect:** Preventing mode collapse to a single token is handled by sampling temperatures and entropy bonuses.
- **Option d is incorrect:** The KL penalty does not make generation deterministic; decoding strategies (greedy vs. nucleus sampling) control determinism.
- **Option e is incorrect:** The candidate in the exam dump mistakenly selected this option. Option (f) is the precise theoretical definition.
- **Option g is incorrect:** Rewarding human preference is the explicit responsibility of the primary reward term $r_\theta(x, y)$, whereas the log-ratio acts as a subtrahend penalty.

Solution to Question 12: Priming in Prompt Engineering

Correct Choice: c (“Providing initial input to guide the model’s response”)

1. Theoretical Rationale & Cognitive Context

In prompt engineering and cognitive psychology applied to generative AI, **priming** refers to the deliberate technique of prepending contextual cues, persona roles, stylistic exemplars, or conceptual prefaces at the beginning of the prompt context window.

- **Attention Conditioning:** By introducing specific semantic tokens into the prefix (e.g., “You are a senior security auditor analyzing C code for buffer overflows”), the multi-head self-attention mechanism activates specialized latent subspaces and associations learned during pre-training.
- **Steering Probability Trajectories:** Priming shifts the conditional probability distribution $P(w_t | w_{<t})$ toward relevant vocabulary, desired formality levels, and specific domain paradigms before the primary user query is processed.

2. Detailed Breakdown of Distractors

- **Option a is incorrect:** Reinforcement learning (e.g., PPO, DPO) modifies model weights through gradient descent during the post-training phase, which is fundamentally distinct from in-context prompt engineering.
- **Option b is incorrect:** Pre-training with additional data refers to continuous pre-training / domain-adaptive pre-training (DAPT).
- **Option d is incorrect:** Adding noise describes adversarial perturbation, stress-testing, or data augmentation.
- **Option e is incorrect:** Clearing or resetting model conversational state is context management, not semantic priming.